

GaussianFormer: per-object adapters and the rasterized-canvas architecture

Week of 8-15 September 2026

Shahaf Valencia Levy

16 September 2026

1. Storage: what an adapter costs in Gaussians

A Gaussian is $14 \text{ float32} = \mathbf{56 \text{ bytes}}$. A LoRA adapter of rank r over a set of linear maps costs $4 * r * \text{sum}(\text{in} + \text{out})$ bytes. Dividing gives every adapter a price in Gaussians, which is the only unit that matters for a compression claim.

per-object payload	params	size	= Gaussians
r=1, attention only	119,808	0.48 MB	8,558
r=4, attention only	479,232	1.92 MB	34,231
r=4, attention + FFN	1,308,672	5.23 MB	93,477
r=16, attention only	1,916,928	7.67 MB	136,923
full fine-tune	196,177,815	784.7 MB	14,012,701
the 20k splat itself		1.12 MB	20,000

Three consequences, all measured:

1. **Rank is the cheap axis, target set is not.** Going r=1 to r=16 costs 16x. Adding the FFN matrices to the attention ones costs 2.7x at equal rank.
2. **Our winning adapter is 4.7x the size of the object it renders.** At the operating point below we spend 6.35 MB to improve a 1.12 MB file.
3. **At equal bytes the rasterizer still wins.** Spending 6.35 MB on Gaussians instead buys about 113k of them, roughly 2.1 dB above where we land.

The honest framing. This is not yet a compression win. It is an *enhancement* win: given a splat someone already produced, we render it better than rasterizing it. Turning it into a compression win requires the adapter to shrink by roughly 10x, which means rank-1 attention-only (0.48 MB, 8.5k Gaussians). That configuration scored -3.65 dB on the old base and has **never been tried on the current one**. It is the single highest-value untested experiment we have.

2. Architecture: what we changed and what it bought

Four changes were tested on a fixed harness: 10 training objects, 30k steps, margins measured as FG-crop PSNR below the rasterizer on the 10 training objects (*fit*) and on 300 unseen objects (*held-out*). Lower is better; negative beats the rasterizer.

change	fit	held-out	verdict
V18 baseline	7.62	20.44	reference
P3: RoPE bandwidth (3 variants)	7.53	20.69	null
P2: projected 2-D RoPE in cross-attention	6.47	19.07	keep
P2 variants (scale, 2-D rays, wider)	6.39	19.29	null

change	fit	held-out	verdict
P1: rasterized canvas as decoder input	6.78	17.47	keep
P1 + P2	6.07	16.95	additive
P1 + P2 + foreground-weighted loss	2.91	17.27	the recipe

Every step is significant under a cluster bootstrap over scenes (paired, 10k replicates). P2 vs baseline -1.38 dB [-1.47, -1.28]; P1 vs P2 -1.60 [-1.69, -1.51].

2.1 Cycles became free

Warm-restarting the same run repeatedly (“cycles”) used to trade fit against generalisation. With the canvas it stops doing so.

chain	fit c1 -> c4	held-out c1 -> c4
P2 + fg	3.35 -> 1.05 -> -0.35	19.39 -> 19.96 -> 20.34 (drift +0.95)
P1 + P2 + fg	2.91 -> 0.47 -> -1.12 -> -2.15	17.27 -> 17.26 -> 17.17 -> 17.06

Both axes improve every cycle with the canvas; the drift is significant without it (+0.95 dB [+0.89, +1.02]). Caveat on the fit column: at 10 objects the interval on P2+fg c3’s -0.35 is [-1.73, +0.93], so that arm does *not* significantly beat the rasterizer. The canvas arm’s -2.15 [-3.25, -1.08] does.

A separate fix was needed to get here: LPIPS evaluated under bf16 autocast destabilised training at low loss (3 collapses out of 3 attempts). Computing it in fp32 fixed it, 0 collapses in 4 chains since.

2.2 It scales

Repeating the comparison with 100 training objects at equal compute:

arm	fit	held-out
baseline	13.40	17.93
P2 + fg	10.51	16.38
P1 + P2 + fg	7.77	11.25 [11.01, 11.49]

The canvas advantage over the identical recipe without it **grows from 2.12 dB at N=10 to 5.13 dB at N=100** [-5.32, -4.95]. For scale, the entire original data sweep moved held-out 20.47 -> 17.93 -> 16.70 going from 10 to 100 to 1000 objects; the canvas at 100 objects beats the 1000-object baseline by 5.5 dB.

3. The result

A shared base (canvas recipe, 3 cycles, 10 objects) plus a **5.23 MB** rank-4 adapter trained on one unseen real scan beats the rasterization of that scan’s own 20k splat:

range	ours	rasterizer	delta
novel random	35.95	35.79	+0.16 (16/24)
novel close	30.20	28.20	+2.00 (8/8)
novel far	38.92	38.39	+0.53 (6/8)
all 40 views			+0.60 , 30/40 won

The per-object full fine-tune reaches +1.82 dB on the same object, but costs 784.7 MB, 150x more, and needs four cycles rather than one.

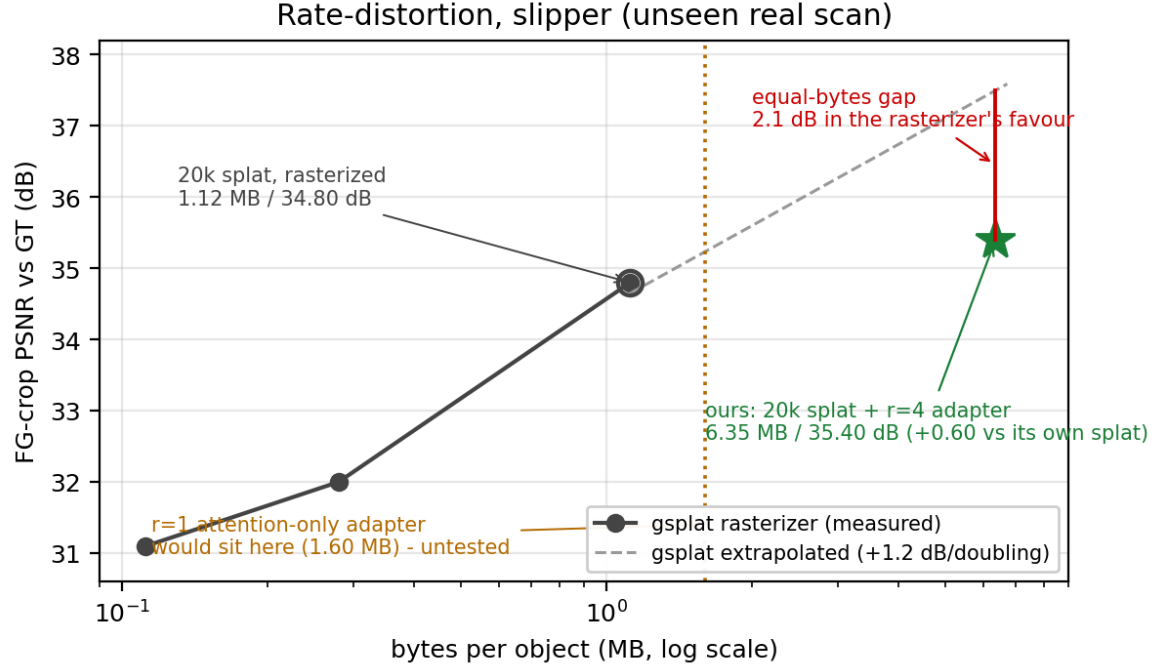


Figure 1: Rate-distortion on the slipper. The green star is our best result; the red bar is what the same byte budget buys if spent on Gaussians instead. The win is real at a *fixed* splat, not at a fixed budget.

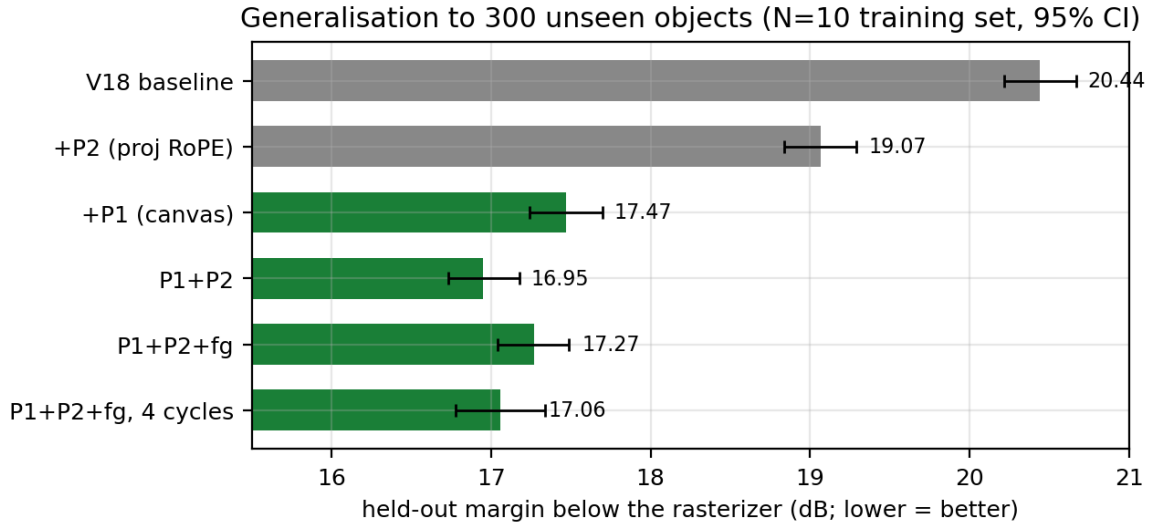


Figure 2: Held-out generalisation by arm, 95% CI. The canvas (P1) is the generalisation lever; P2 is the fit lever.

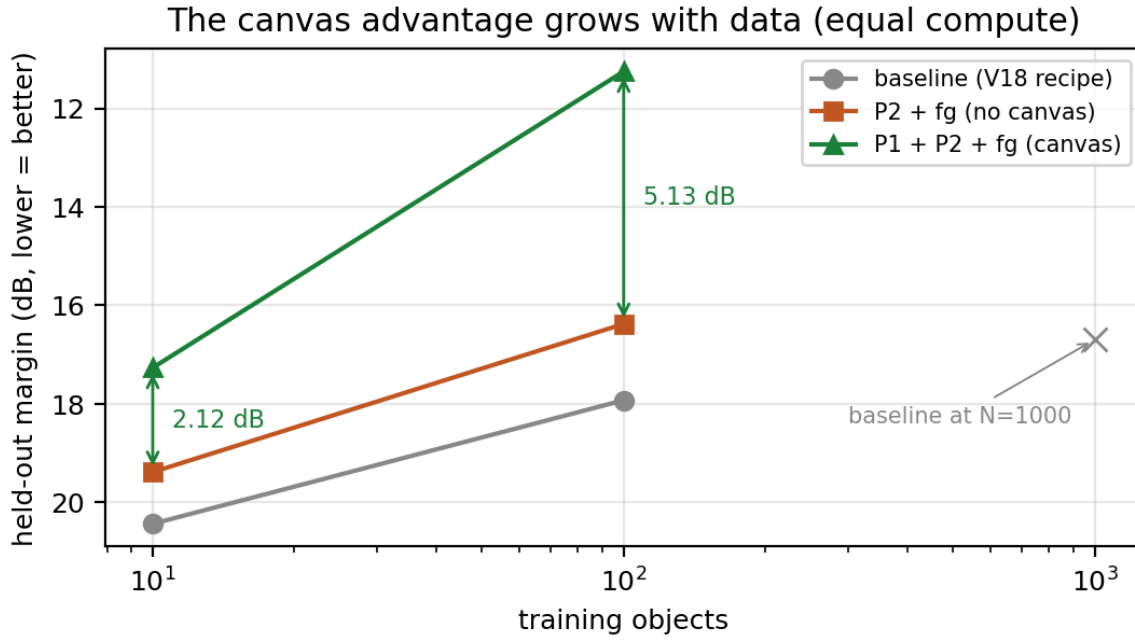


Figure 3: The gap widens with data, which is the argument for a full-dataset run.

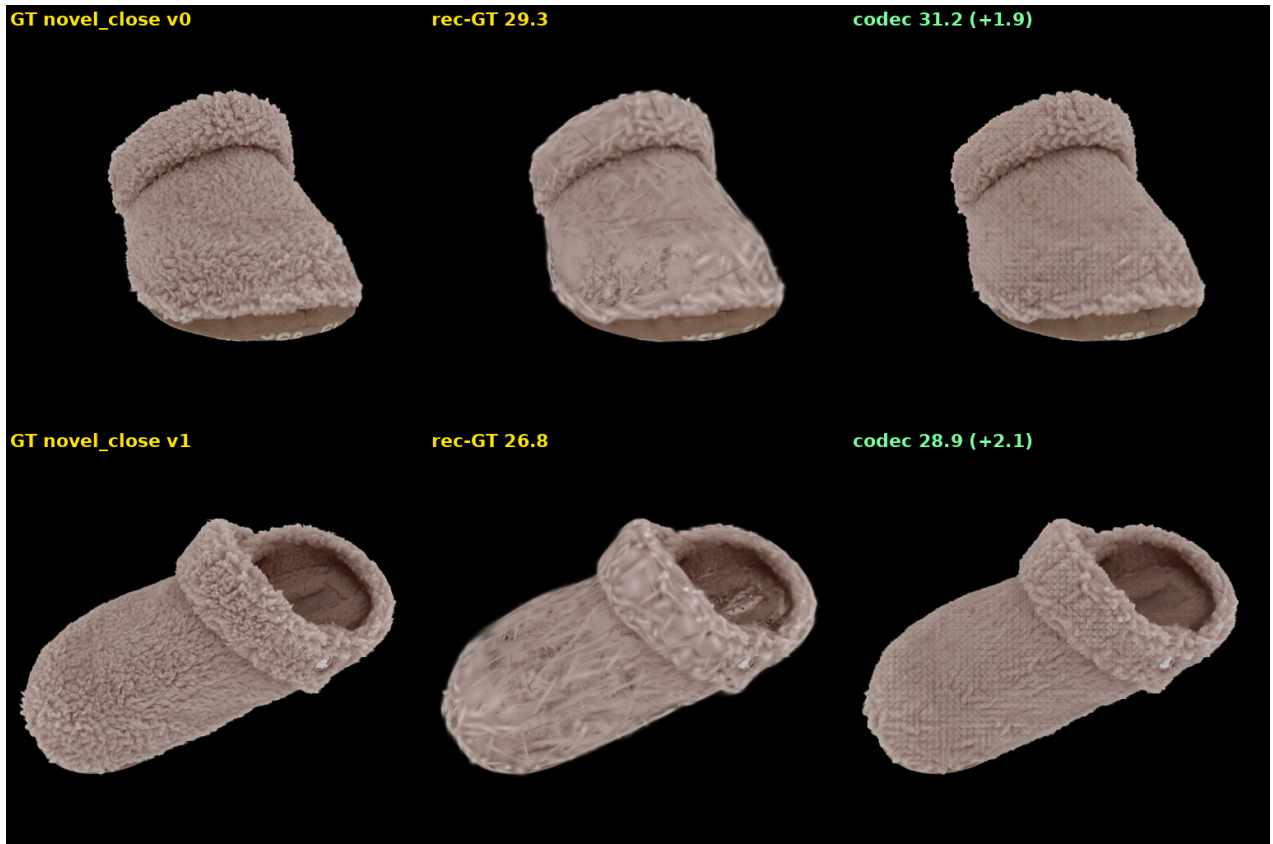


Figure 4: Slipper, unseen by the base, the two close-range views. Left ground truth, middle the rasterized 20k splat, right ours. The rasterizer smears the fleece into streaks; the adapter keeps it as texture. Full six-view figure in docs/figures/.

Which base matters, and it is not the one that fits best. Adapter quality tracks the *base*’s held-out margin, not its fit:

base	base fit	base held-out	adapter result
P2+fg	3.35	19.39	-0.33
P2+fg c2	1.05	19.96	-0.44
P2+fg c3	-0.35	20.34	-0.50
P1+P2+fg c3	-1.12	17.17	+0.60

Fitting the base harder makes the adapter worse. Reproduction on a second object is running at the time of writing.

4. Two honest caveats

The rasterizer cannot be pruned after training. Feeding the winning model a black canvas at inference collapses it to worse than its own starting checkpoint. Feeding it the right object from an adjacent camera is barely better.

canvas fed	fit	held-out	model PSNR
normal	-1.12	17.17	27.80
right object, wrong camera	27.80	29.42	15.54
all black (arm removed)	30.15	32.49	12.48

So the deliverable is splat + base + adapter + a rasterizer in the loop, and the canvas is consumed pixel-aligned rather than as a generic prior. Inference cost is the rasterizer (about 1 ms) plus the transformer (about 476 ms per view).

It is not copying the canvas, though. Measured directly: on unseen objects the output sits 18% away from the canvas it was given (27.5 dB from it) and every pixel is synthesised. A deliberately built control that *does* add its output to the canvas as a residual was tested and rejected: it never beat the rasterizer on any unseen object in any regime, and per-object training made it worse rather than better. Its apparent strength was a constraint, not learning.

The accurate description is therefore: **a learned, view-aligned refiner of a rasterization that synthesises every pixel it emits.** Not a standalone neural renderer.

5. Proposed next steps

1. **Rank-1 attention-only adapter on the canvas base.** 0.48 MB, 8.5k Gaussian-equivalents. If it holds most of the +0.60 dB this becomes a genuine rate-distortion result rather than an enhancement one. One day.
2. **Full-dataset run with the canvas recipe.** The N=10 to N=100 trend is the justification; about three days on one node.
3. **Canvas dropout during training.** Feed a black canvas on a fraction of steps so the scene-token path stays alive, and measure whether the two paths can coexist.
4. **Patch-grid artefact.** The model’s error spectrum spikes at exactly 8 px, the patch size, while the rasterizer’s is broadband. A zero-initialised 9x9 residual convolution on the decoder output (732 params, warm-starts from any checkpoint) is training now.

Code is on branches `exp/p1-canvas` (main line), `exp/p1-residual` (rejected control, kept for the write-up), `exp/p4-deblock`. All results reproduce from `data_v10/probe_report.py` and `data_v10/probe_ci.py`.